Intro To Design Patterns

COMP3522 OBJECT ORIENTED PROGRAMMING 2

WEEK 9

What are Design Patterns

Common design solutions to common architectural problems.

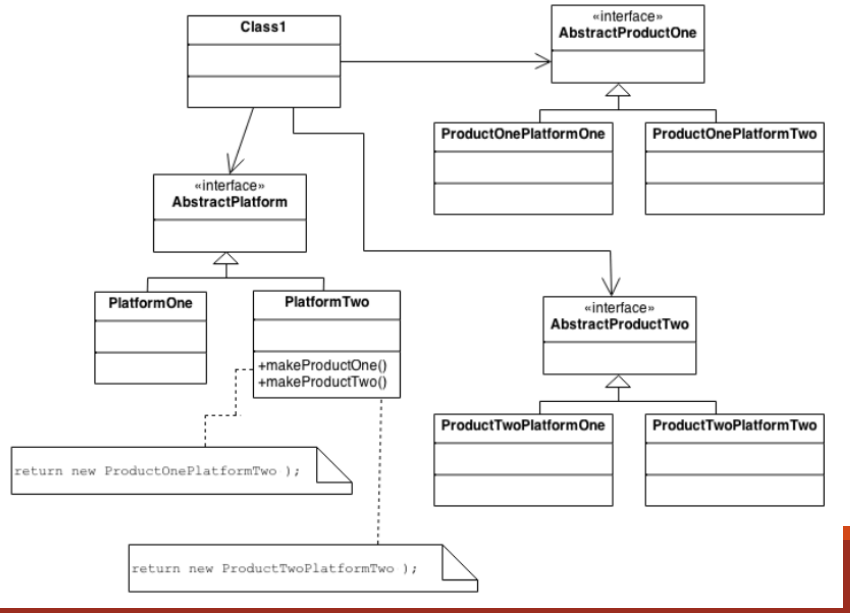
How can I write systems so:

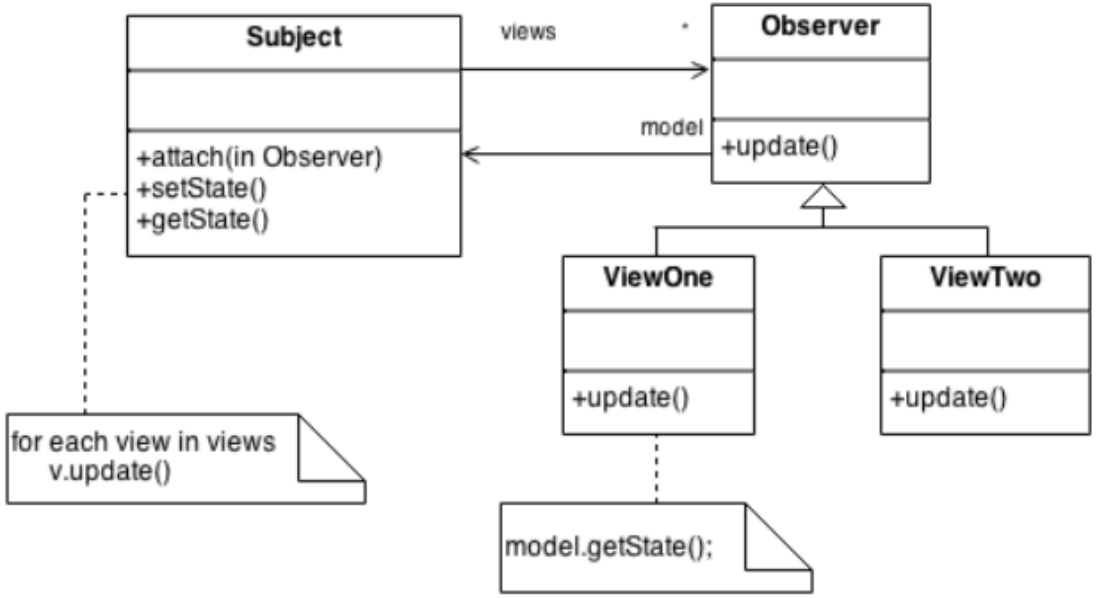
- Classes can **communicate** with each other with low coupling?
- Classes can be combined to form new **structures?**
- Classes can be **created** with different strategies and techniques?

What are Design Patterns

Design patterns are not an API or magic function we call to solve design issues

Design patterns describe how to structure our code/classes to solve some specific design issue





Design Patterns - Advantages

- Don’t re-invent the wheel, use a proven solution instead
- Are abstract and can be applied to different problems
- Communicate ideas and concepts between developers
- Language agnostic. Can be applied to most (if not all) OOP programs.

Design Patterns - Disadvantages

- Can make the system more complex, making the system harder to maintain. Patterns are deceptively ‘simple’.
- The system may suffer from pattern overload.
- All patterns have some disadvantages and add constraints to a system. As a result a developer may need to add a constraint they did not plan for.
- Do not lead to direct code re-use.

Categorizing Design Patterns

- **Behavioural**

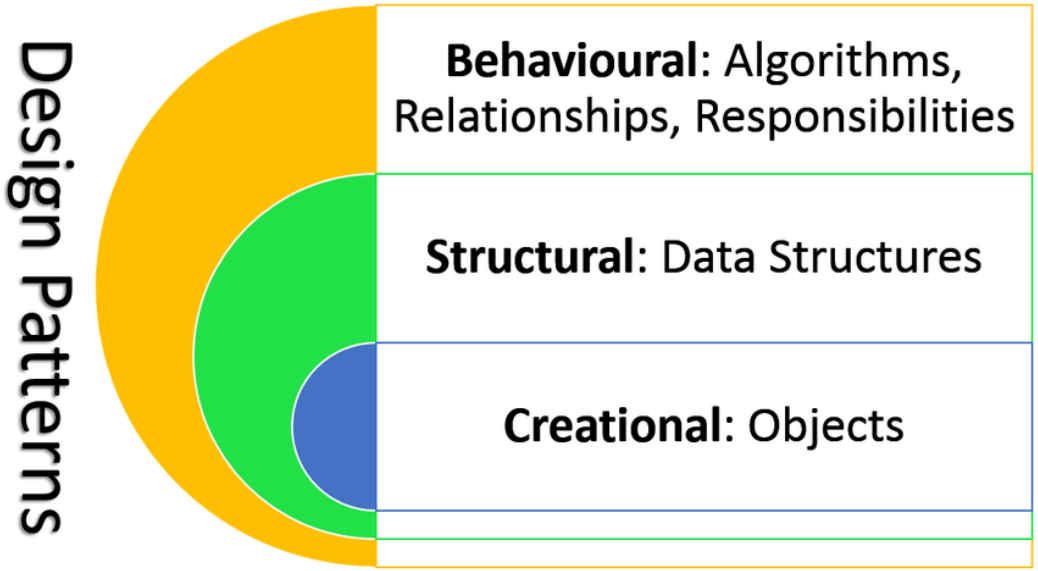
Focused on **communication** and interaction between objects. How do we get objects talking to each other while minimizing coupling?

- **Structural**

How do classes and objects combine to form new **structures** in our programs? Focus on architecting to allow for maximum flexibility and maintainability.

- **Creational**

All about class **instantiation**. Different strategies and techniques to instantiate an object, or group of objects



Picking a Pattern

Step 1

- **Understand the problem** you are facing in terms of dependencies, modularity and abstract concepts.

Step 2

- **Identify** if this is a behavioural, structural or creational issue?

Step 3

- Are there any **constraints** that I need to follow?

Step 4

- Is there a **simpler solution** that works? **If not, pick a pattern.**

Singleton

WHEN ONE IS ENOUGH

Design pattern: a really easy one!

Design issue: How can I have a **globally accessible object**, where I ensure that **only one instance** of this object exists?

Solution: **Singleton Design Pattern**

Singleton pattern

Singleton Design Pattern is a Creational design pattern

Guarantees that only a **single instance** of a class will ever exist

We want to **prevent more than one copy from being constructed**

We must write code that enforces this rule

Singleton pattern requirements

1. **Instantiates** the object on its first use
2. **Ideally hides** a private initializer
3. **Reveals** a public `get_instance` method that returns a reference to a static instance of the class
4. **Provides** “global” access to a single object

Why/how do we use it?

Use the singleton pattern **when you need to have one and only one object of a type** in a system.

Singleton is a **globally accessible class** where we guarantee **only a single instance** is created

That's it.

Really, that's all there is to it.

Code sample (so easy!)

```python
class MySingleton:
    __instance = None

    @staticmethod
    def get_instance():
        if MySingleton.__instance is None:
            MySingleton()
        return MySingleton.__instance

    def add_num(self, n):
        MySingleton.__instance.data += n

    def __init__(self):
        if MySingleton.__instance is not None:
            raise Exception("This class is a singleton!")
        else:
            MySingleton.__instance = self
            MySingleton.__instance.data = 0
```

```python
s = MySingleton.get_instance()
s.add_num(6)
print(s, s.data)

s1 = MySingleton.get_instance()
s1.add_num(3)
print(s1, s1.data)

s2 = MySingleton.get_instance()
s2.add_num(2)
print(s2, s2.data)
```

singleton_example.py

Application – Game screen management

Game has multiple screens

- Start, gameplay UI, game over, store, etc

Different screens must be displayed at various places in the code

- Store class wants to show store screens
- Gameplay logic wants to show start/gameplay/game over
- Settings logic wants to show settings screen

Need a **central global place** to call to **load specific screens** on demand

Application – Game screen management

Create enums for every screen type: `MAIN_MENU`, `GAMEPLAY`, `GAME_OVER`

Create Screen classes for different screens: MainMenu, Gameplay, GameOver

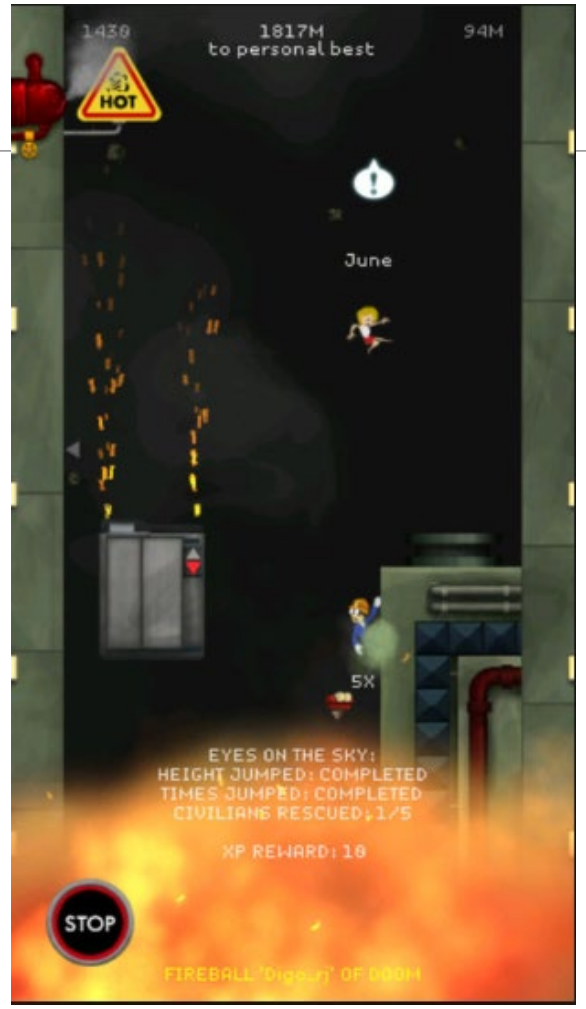
Create a **ScreenManager singleton** that has**:**

- dictionary of `ScreenEnum` to `Screen`
- Responsible for calling correct `screen` based on the `ScreenEnum`

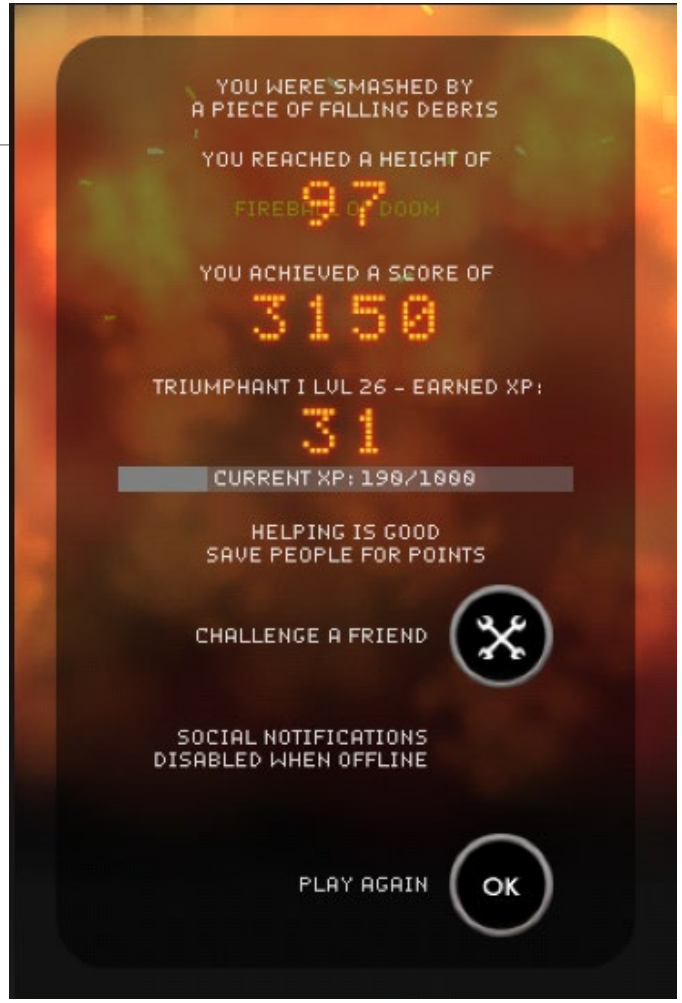
Have a method `show(screen_enum)` that accepts a `ScreenEnum` and shows a screen from a dictionary to the screen

Mechanic Panic – Singleton screens example

ScreenEnum enum: MainMenu
ScreenManager.getInstance().show(MainMenu);



ScreenEnum enum: Gameplay
ScreenManager.getInstance().show(Gameplay);



ScreenEnum enum: GameOver
ScreenManager.getInstance().show(Gameover);

singleton_example_screens.py

Singleton Pattern: Advantages

- Great for systems where there is logically **only one of something**
- Designed to **prevent unnecessary duplication**
- Candidate for Lazy Initialization. Object only **instantiated the first time it's needed**.
- **Centralized state management**. All parts of the program use the same shared state

Singleton Pattern – Disadvantages

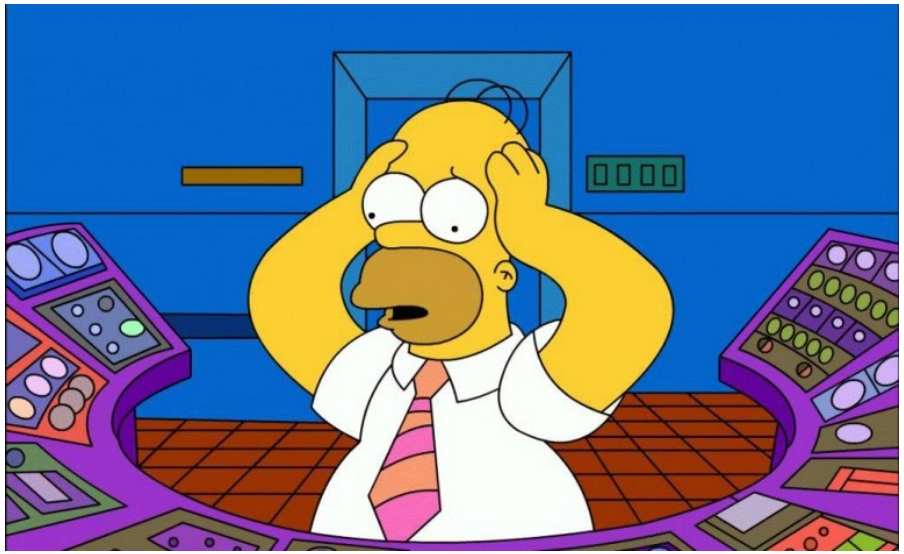
- Singleton acts as a global variable, **potentially leading to tight coupling**
- Potentially violates single responsibility principle if **Singleton performs too many operations**
- **Race conditions** in multithreaded programs
- The global nature of the pattern can **lead to overuse**

OBSERVER

Introduction

I want to create a system where **one object** can **broadcast information** to **multiple objects**

How do I notify a bunch of different kinds of objects if the state of one part of the system changes without **tightly coupling** that part of the system with the rest?



Introduction

We like to partition our systems into cooperating classes

Those classes share information

We need to maintain consistency

But we can't couple them tightly because that reduces their flexibility

We use an idiom you will see often in programming called Publish-Subscribe:

- The **Subject** publishes notifications without knowing specific **observers**
- Any number of **Observers** can subscribe to receive notifications*

* Sounds a little like Java GUI listeners to me!

Observer design pattern

Design pattern category: Behavioral

The **Observer pattern** describes how to establish these relationships

There are two key objects:

1. **Subject** may have any number of dependent observers
2. **Observers** are all notified whenever the subject undergoes a change of state

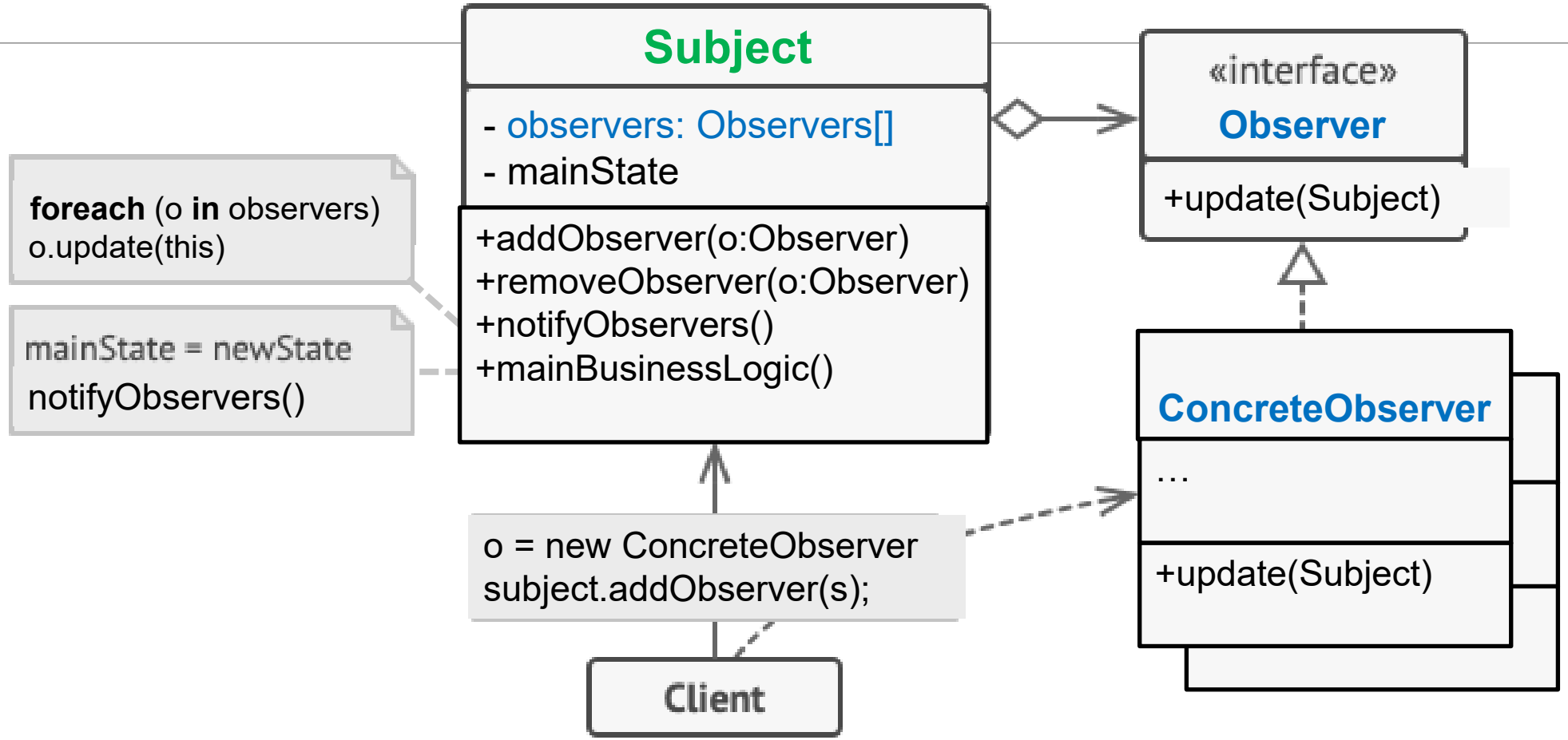
Each observer queries the subject to synchronize their states

This pattern ensures that when a subject changes state all its observers are automatically notified

Observer design pattern

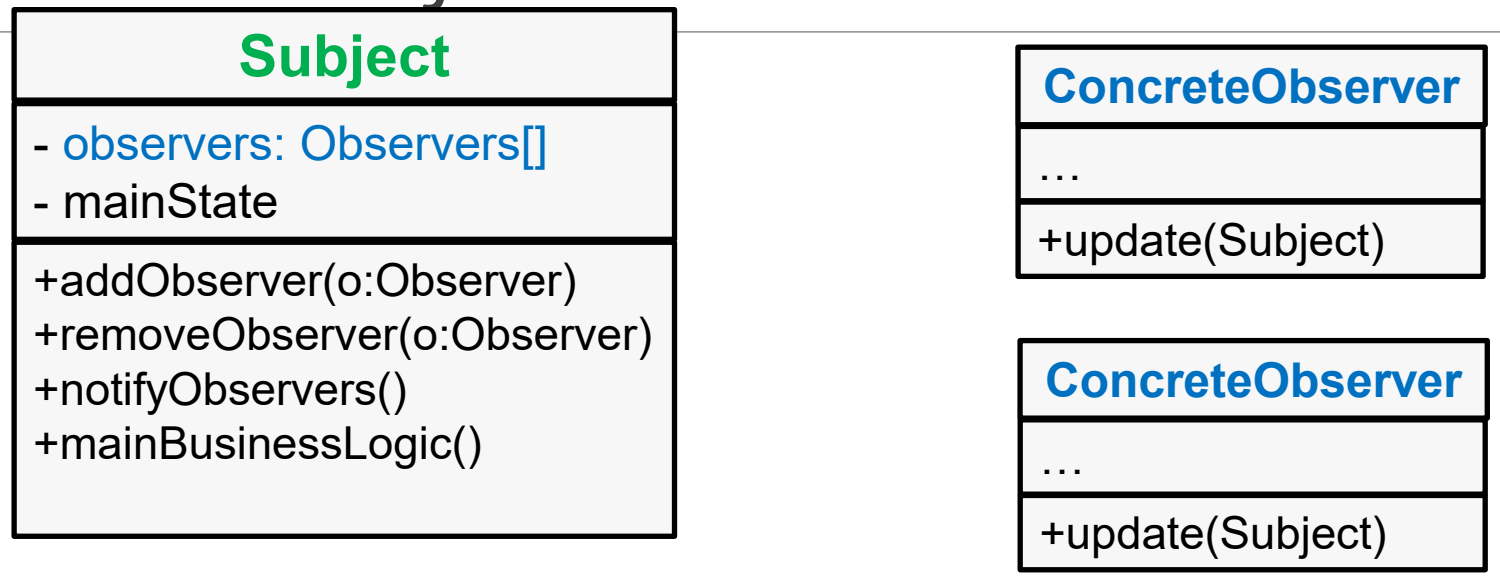
1. (Abstract) **Subject**
   - Knows its Observers
   - Any number of Observers may observe a subject
   - Provides an **interface for attaching and detaching** Observers
2. (Abstract) **Observer**
   - Defines an **updating interface** for objects that should be notified of changes in a subject
3. **ConcreteSubject**
   - Sends **notification** of its changed state to its observers
4. **ConcreteObserver**
   - Maintains a reference to a ConcreteSubject object
   - Stores state that needs to be consistent with the subject's
   - Implements the Observer updating interface

Observer design pattern: Class diagram



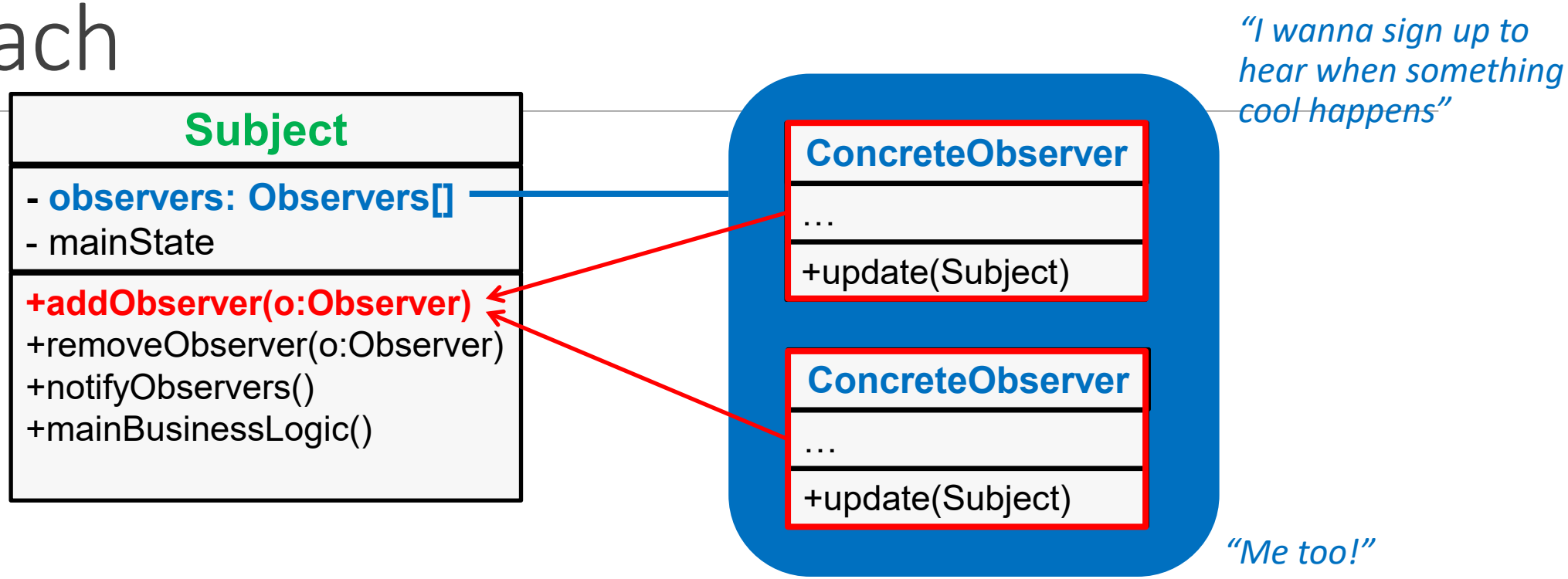
https://refactoring.guru/design-patterns/observer

1. Create subject and observers



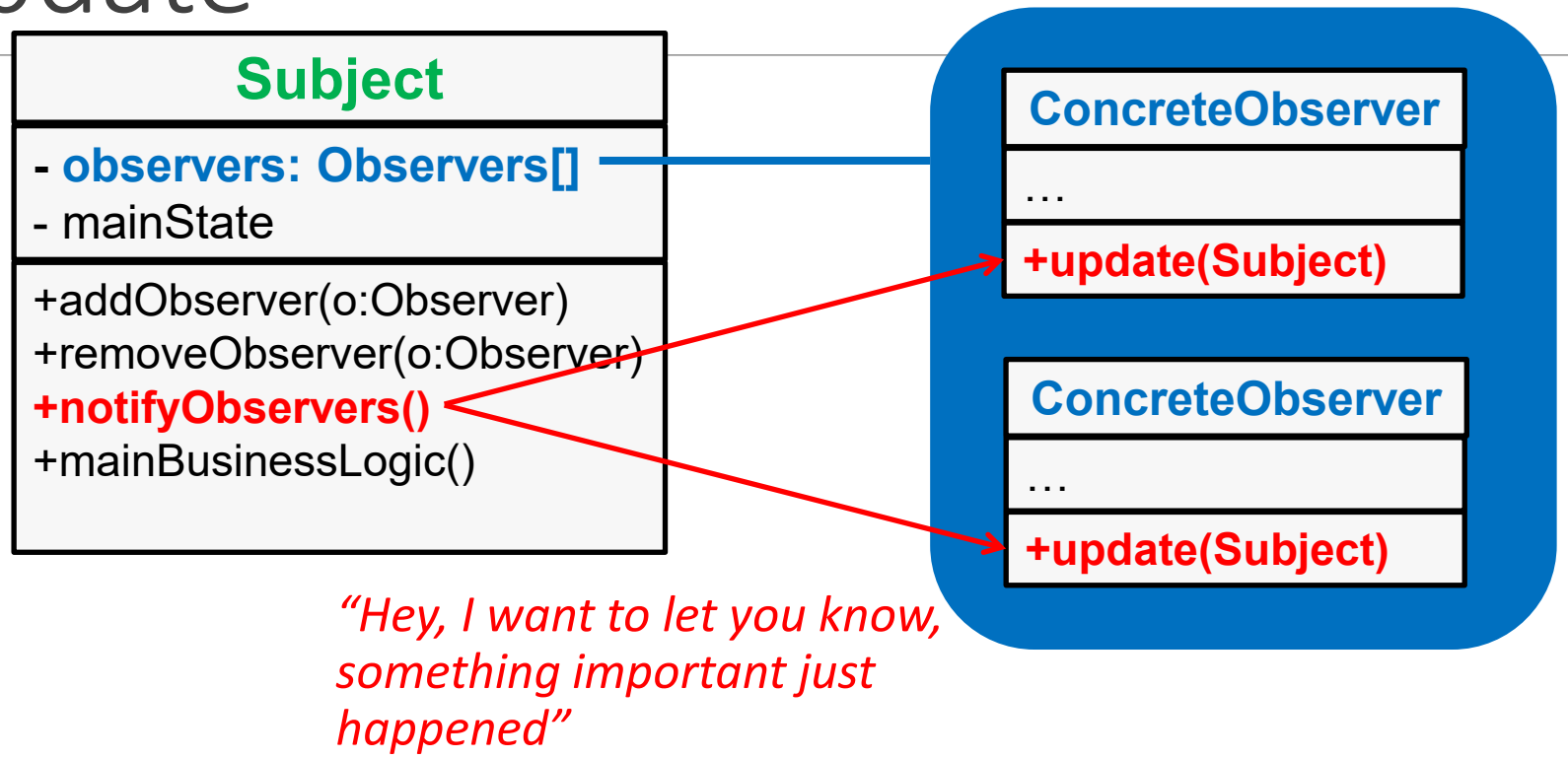
Instantiate subject, and 2 observer objects. Notice how all objects are separate. The objects have no knowledge of each other yet.

2. Attach



Observer objects can be added to a Subject to "listen in" to important events

3. Update



Subject notifies observer objects when important event happens

Use cases

Use the Observer pattern when:

- A **change to one Subject** requires **changing others**, and you don't know how many objects needs to be changed
- An **object** needs to **notify other objects** without making any assumptions about their concrete type (loose coupling!)

I need the professor to be **notified** when a student joins his/her class

I want the display to **update** when the size of a window is changed

I need the schedule view to **update** when the database is changed

News example

Design issue: a Newspaper needs to notify iPhone and Android observers when there is new news

For example:

Newspaper gets new news that *"it's snowing today!"*.

This news is broadcast to all iPhone and Android observers. Each observer will respond that they've gotten the news.

Ie: *iPhone user Jeff got the news: It's snowing today!*

Android user Eric got the news: It's snowing today!

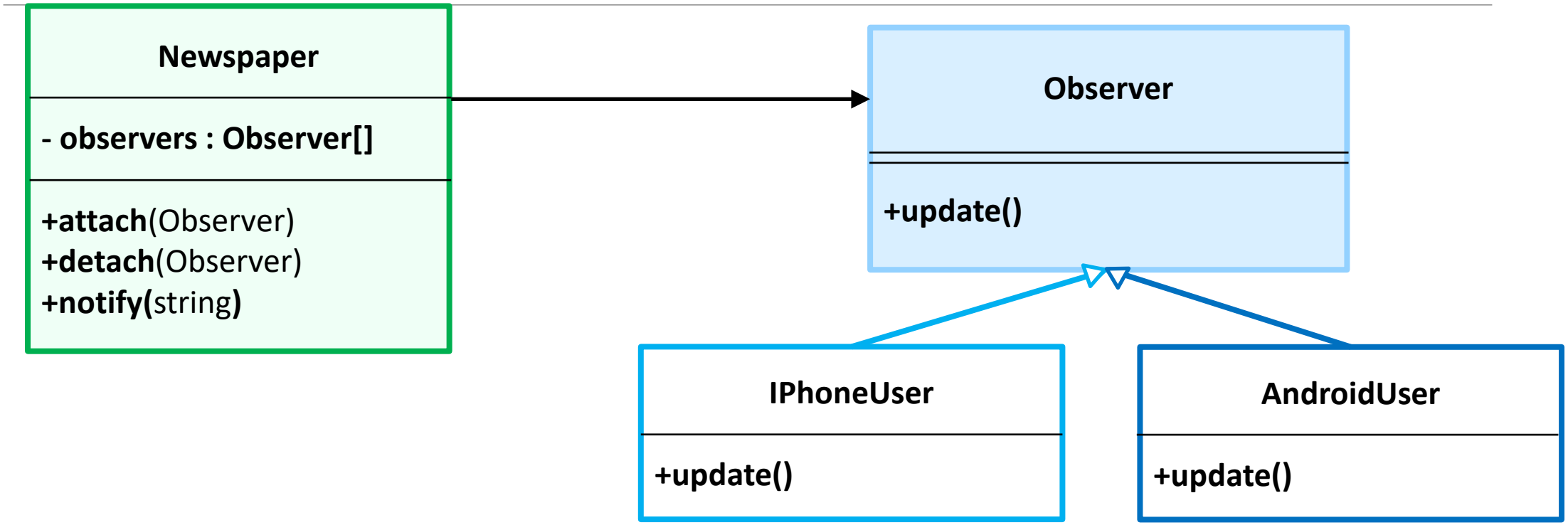
Newspaper class is a **Subject**

- Subject contains a list of observers
- Subject's **notify()** will call all observers' **update()**

iPhone and Android class are **Observers**

- All observers have an **update()**
- Perform own logic when this **update()** called by **Subject**

News example



observer_pattern.*py*

Main

#create iPhoneUser, AndroidUser, Newspaper
#attach iPhoneUser, AndroidUser to Newspaper
#call Newspaper's notify() with string

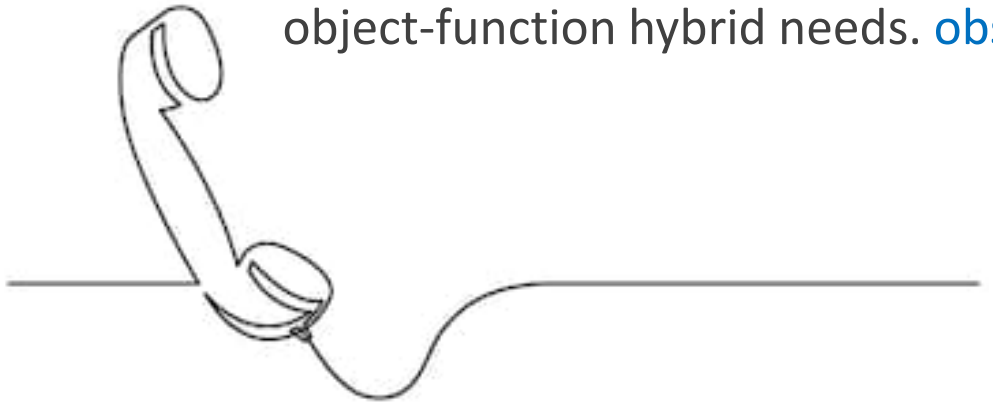
Callable Objects

We know functions can be treated as objects.

What if we wanted to treat objects as functions?

We can totally do that!

Just implement the `__call__(self)` method and add any parameters your object-function hybrid needs. observer_pattern_callable.*py*



Observer Pattern: Advantages

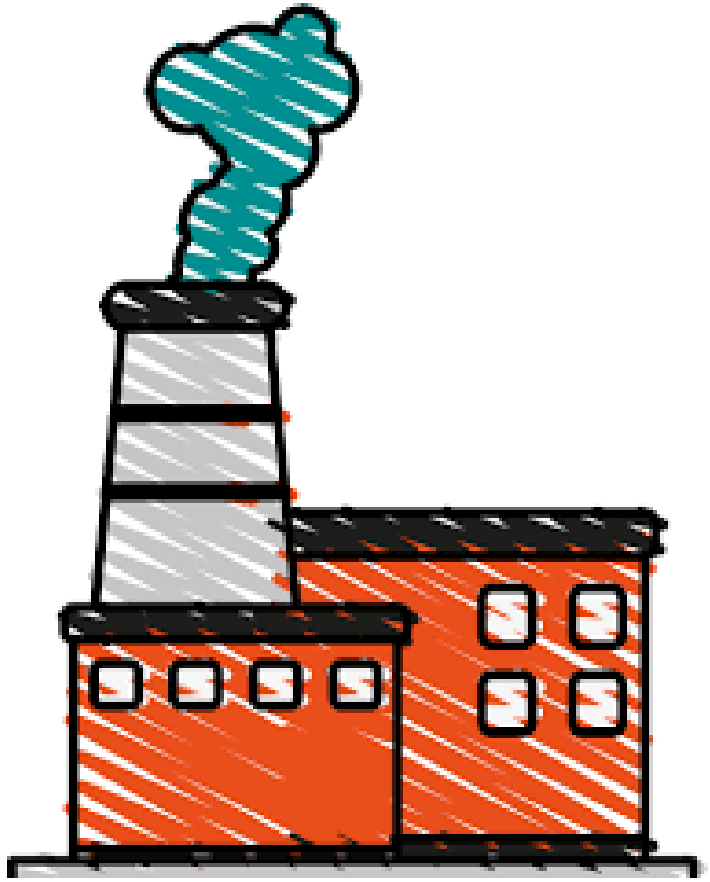
- **Loose coupling** where the Subject doesn't know the concrete Observers
- Follows Open/Closed principle where **new observers can be added without modifying Subject**
- **Event driven architecture** separates producers from consumers
- Single responsibility where each **Observer handles their own response to updates**

Observer Pattern – Disadvantages

- **Hard to predict exact order of calls to Observers** when an update occurs. Especially troublesome if order of observer calls matter
- Performance issues with many Observers. Each update MUST call every Observer's update method
- Performance and memory issues if **observers are not removed properly**

Factory Pattern

First let's clarify the word **Factory**…



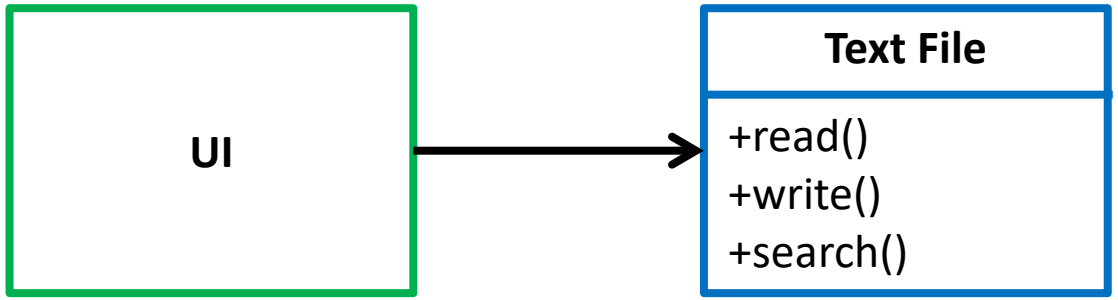
- Something that is responsible for making things
- In this case objects.
- The word **factory** is thrown around a lot, don't mix it up with the **Factory Pattern** or the **Abstract Factory Pattern**. They are also responsible for making objects but are different.
- A **Factory** could refer to:
  - The pattern(s)
  - A class that creates objects
  - A static/class method that generates objects.

Some more lingo – **Creation Method**

- A **method or function that is responsible for creating an object**. It is a factory.
- Different from the initializer, although you can argue that an initializer is a creation method too. A **creation method can be a wrapper** around an **initializer call**. (Check the example below)
- A creation method doesn't have to make a new instances of a class, it can return a cached object or even re-use objects from a collection.
- Resist the urge to call it a **Factory Method**. The **Factory Pattern** is also known as the **Factory Method Pattern** and it can cause all sorts of confusion.

```
class Asteroid:
    …
    @classmethod
    def generate_random_asteroid(cls, min_speed, max_speed):
        speed = random.randomint(min_speed, max_speed)
        asteroid = cls()
        asteroid.speed = speed
        return asteroid
```

Remember the dependency inversion principle?



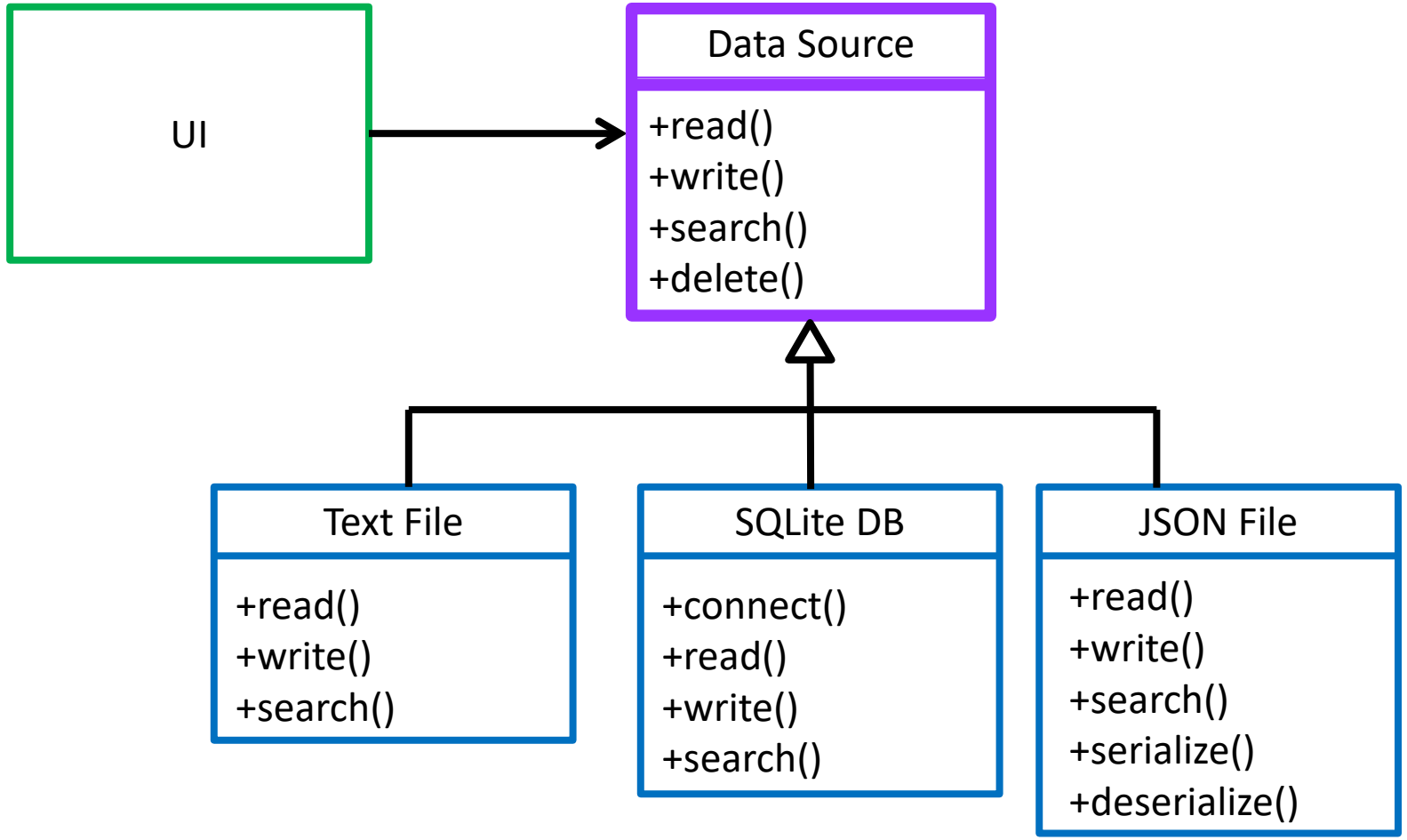
Say we were developing an app with a UI that was populated with some data from a text file.

After a few releases and years of development, for some reason we decide to switch out to a more secure source of data , perhaps an encrypted JSON file or even perhaps a SQLite Database. We would have to edit all the modules/classes that dealt with our app's UI!

Remember the dependency inversion principle?

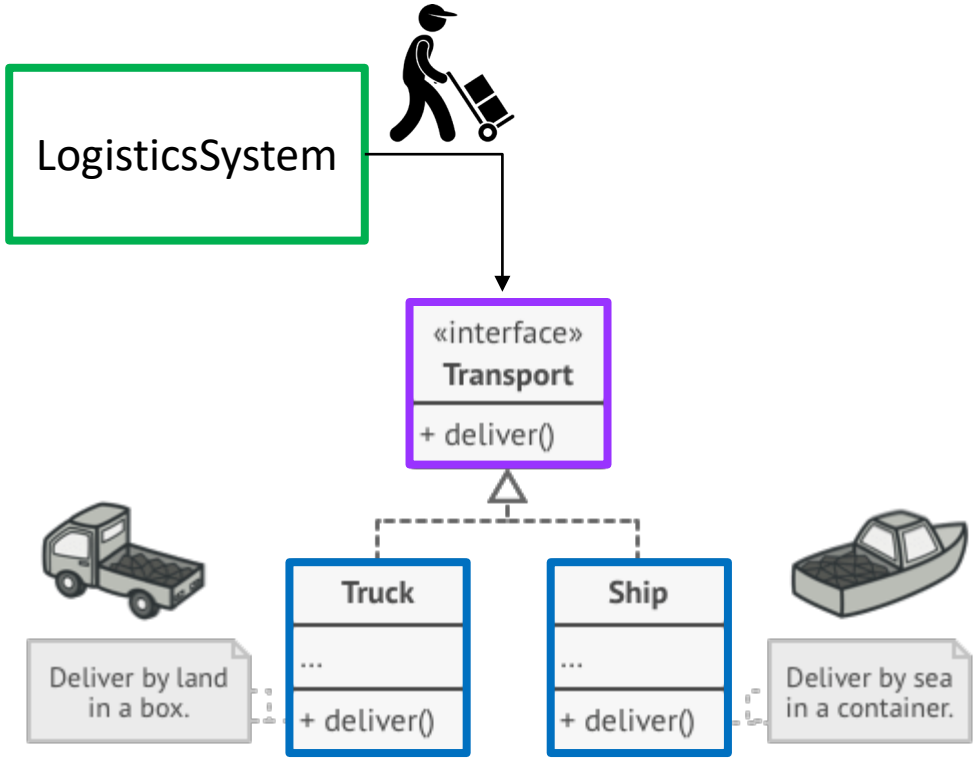
We can decouple our system to instead depend on a **Data Source** which is an **abstraction** that **hides** different data sources.

Our UI would just have to be **dependent on a common interface** provided by the **Data Source** and not be concerned with how that data source is implemented



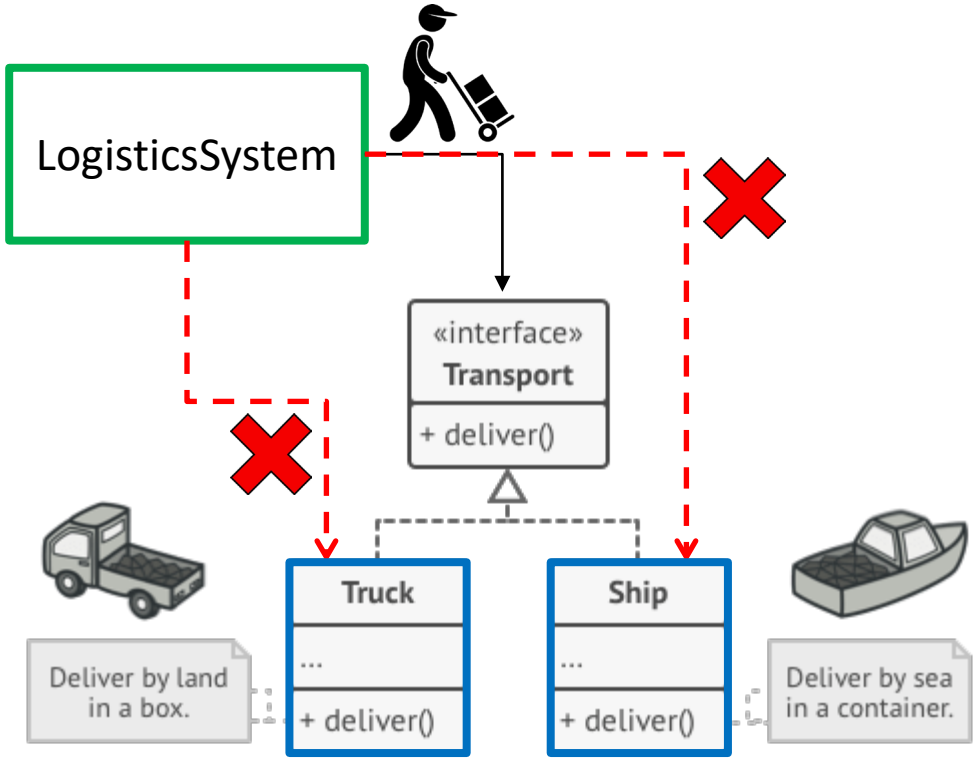
Factory Pattern

- A way to **decouple client code** that **uses** an object or a service, from the code that **creates** an object or a service.
- Consider a scenario where a system is managing the logistics of delivering goods.
- If inheritance is done correctly, all a **Logistics System** cares about is depending on the **Transport Interface**, not the **concrete transports** (**Truck** or **Ship**). This follows **the Dependency Inversion Principle**.



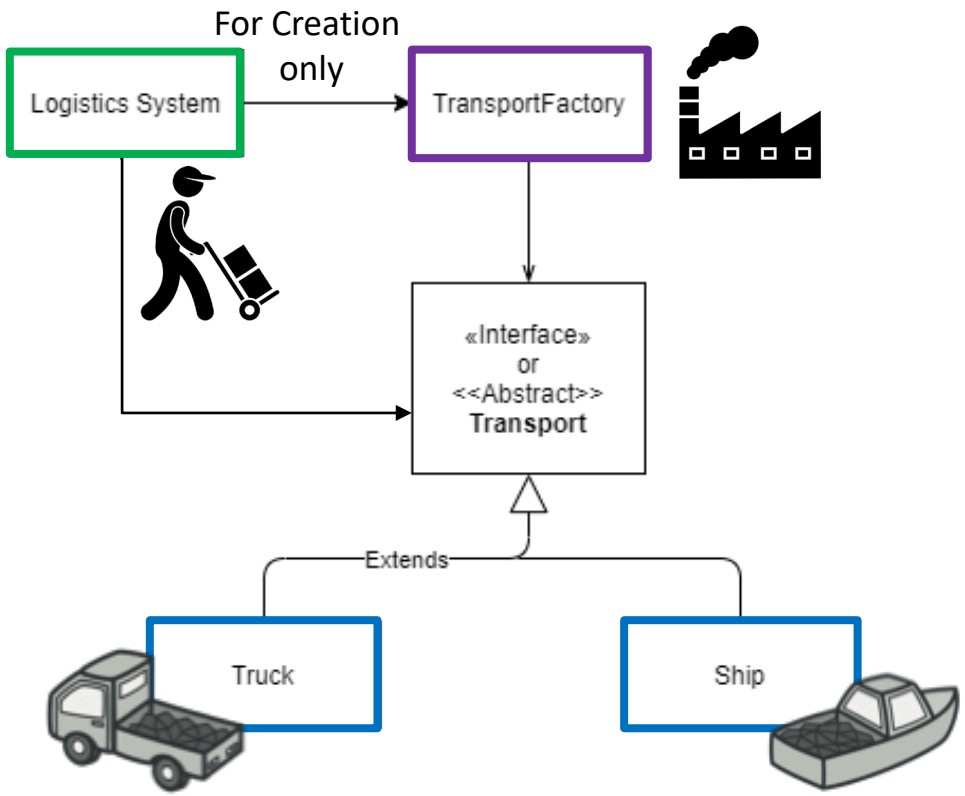
Factory Pattern

- But at some point we need to create the **concrete class**. If the **Logistics System** did that, then it would need to call the Truck() or Ship() initializers.
- This would make **Logistics System dependent on the concrete classes** and break the Dependency Inversion Principles. A high-level module is now dependent on a low-level module.



Factory Pattern

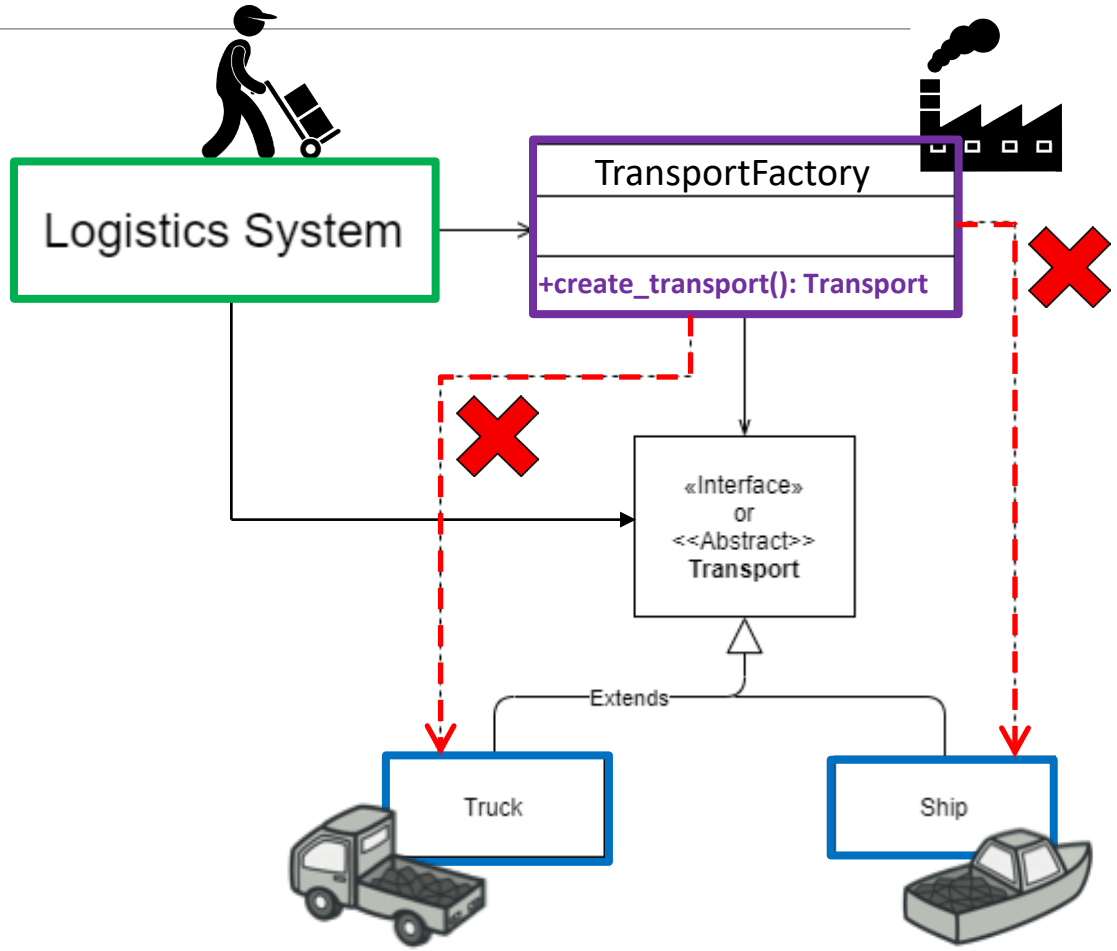
- So when it comes to creating these objects we need another entity to handle the responsibility of creating a specific type of **Transport**.
- We need a **factory** which will create a concrete **Truck/Ship** and return it to the **Logistics System**.
- As far as the **Logistics System** is concerned. It doesn't necessarily know that it is receiving a **Truck/Ship**. All it needs to know is that it is receiving a **Transport**.



Factory Pattern

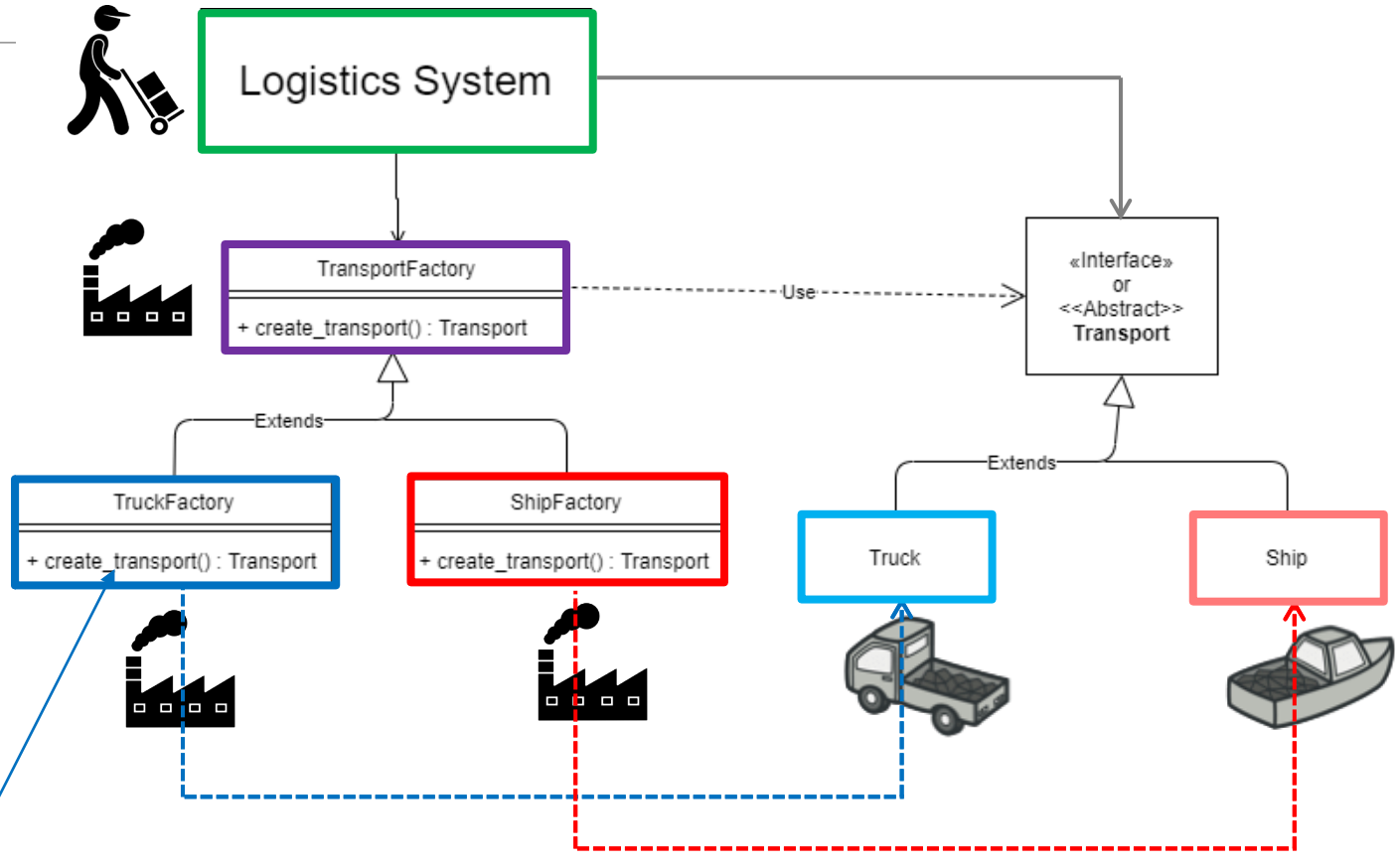
- This still isn't enough. All we have done is move the call to our **Truck()** or **Ship()** initializer to the **TransportFactory**. This isn't that useful.
- Say our **TransportFactory** has a create_transport() method. Now this method will have a massive **if-else** or match statement block where it will be dependent on the concrete Truck and ship Classes

```
def create_transport(self) -> Transport:
        if self.current_type == Ship:
                return Ship()
        else:
                return Truck()
```



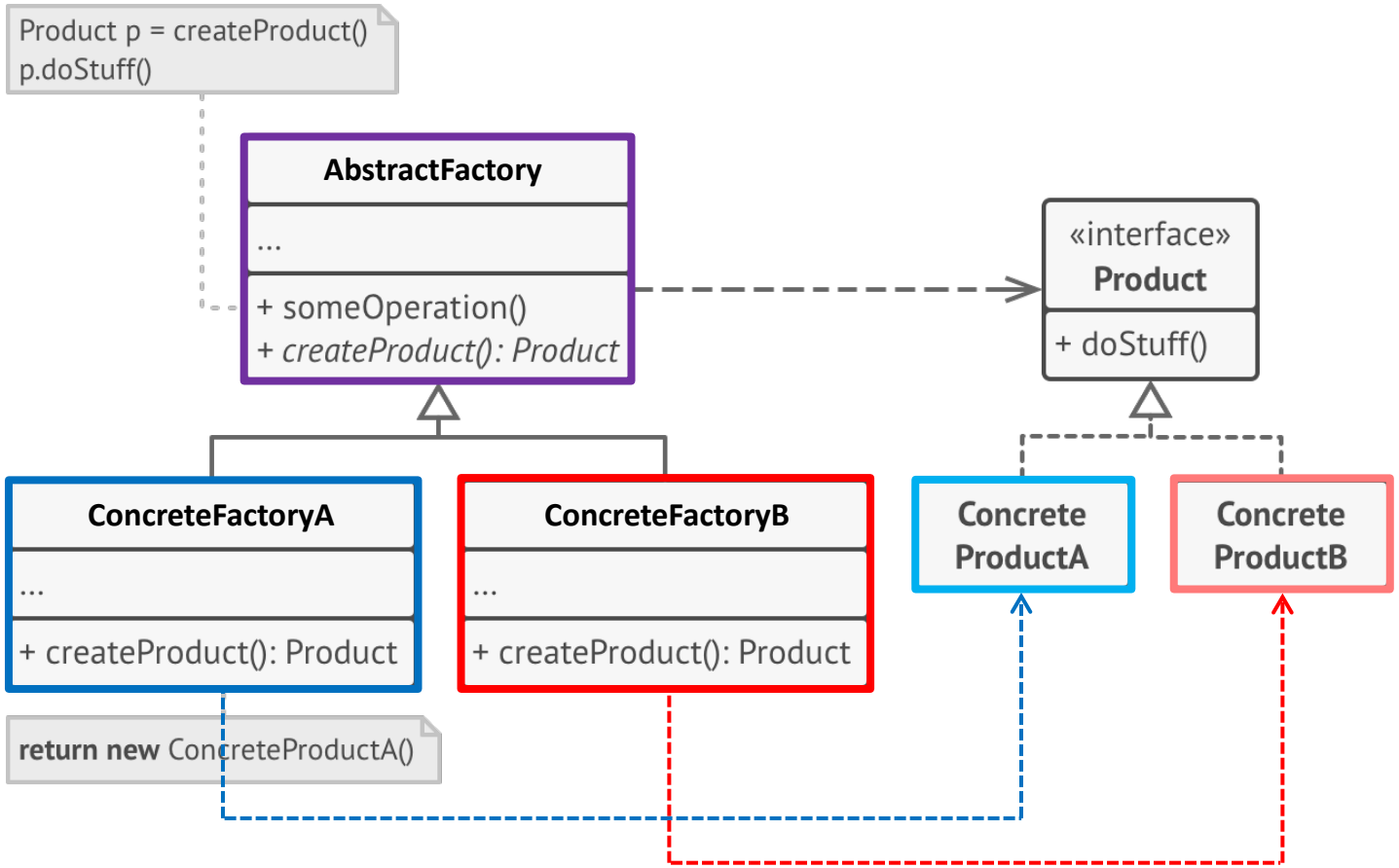
Factory Pattern

- That's better.
- We can simply inherit from **TransportFactory** and override the **create_transport()** method.
- All we need to do is supply the **Logistics System** with the right factory. (Which can be done by a controller).
- As far as the **Logistics System** is concerned it needs an object with the **TransportFactory** interface.




```
def create_transport(self)-> Transport:
    return Truck()
```

Factory Pattern



Let’s Build a Factory Pattern Together.

Let’s say we need to implement a Factory Pattern for a website hosting a forum messaging board. This website creates a new user object when logging in: views the forum as a guest or signs up to be a member.

Our system can deal with two kinds of users. A **Guest** and a **Member**.

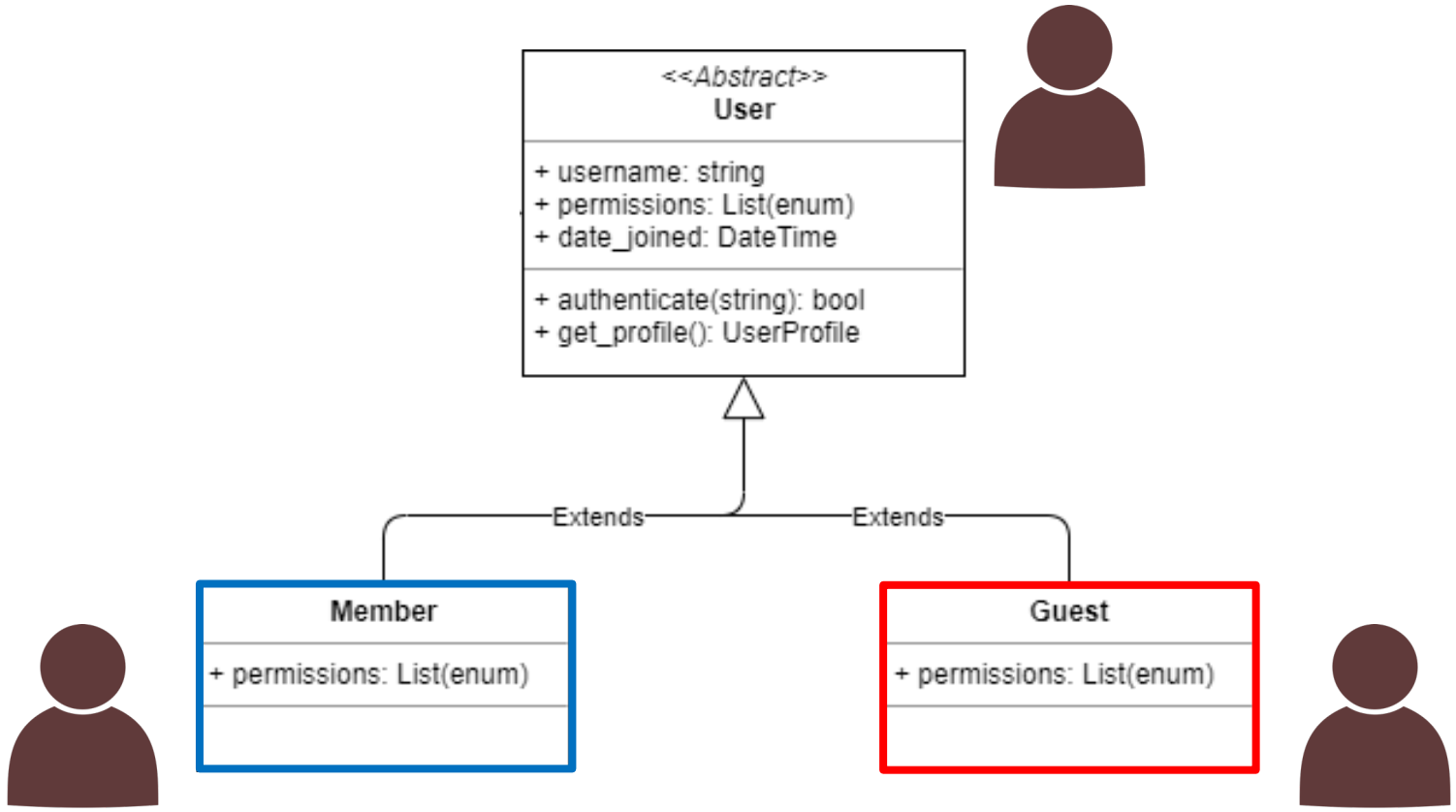
A **Guest** has the following **permissions**:

- Read Posts
- Like
- Share Posts
- Flag Posts

A **Member** can do everything a **Guest** can, but they can also **write** posts.

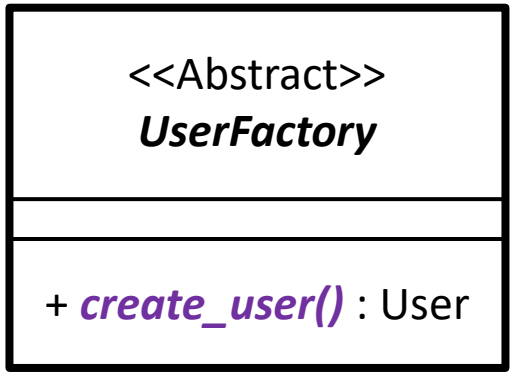
Step 1: Let’s create a UML Class diagram depicting **the User hierarchy.**

Step 1: Define a product hierarchy



Step 2: Define a factory class

- Let's create a **UserFactory** class.
- Specify the **creation method**.
- In the Factory Pattern, a Factory class doesn't need to solely be responsible for creating objects. It can be a larger class with a creation method.
  - For example a UserFactory class might also keep track of how many user accounts are currently active.
- **The base factory doesn't need to be abstract**. It can create a default product. In our case, let's make it abstract though



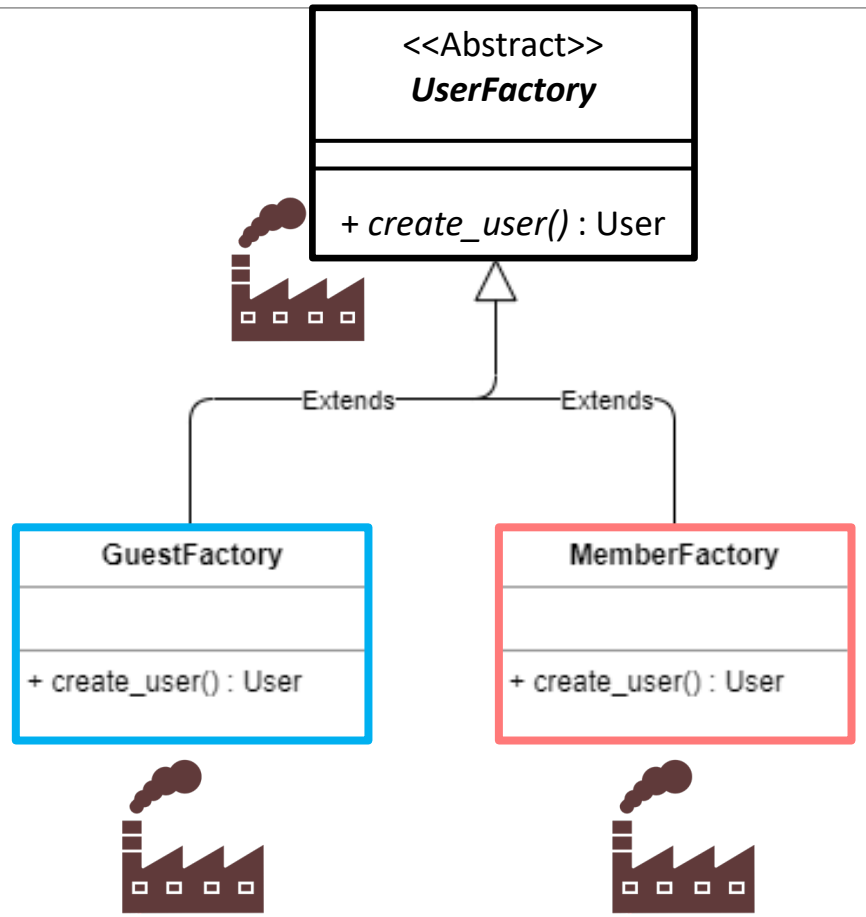
Step 3: Create a Factory hierarchy and override the creation method.

We want to create our concrete factories in this step.

Each concrete factory will create a new object of its corresponding user type and return it.

- **GuestFactory** creates Guests

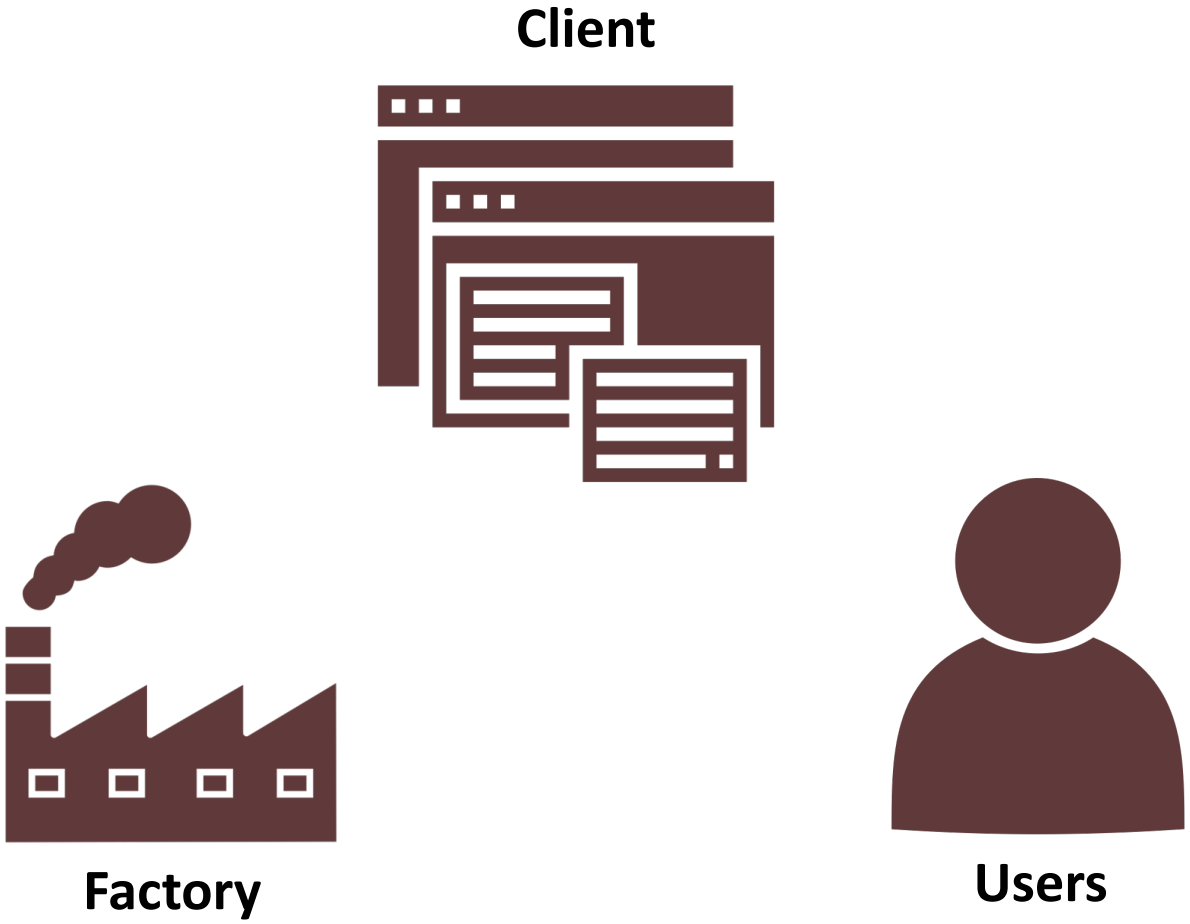
- **MemberFactory** creates Members



Step 4:
Integrate it into our Forum Website

Bring it all together.

Add a client class, and mark the associations between Client, Factories and Users.



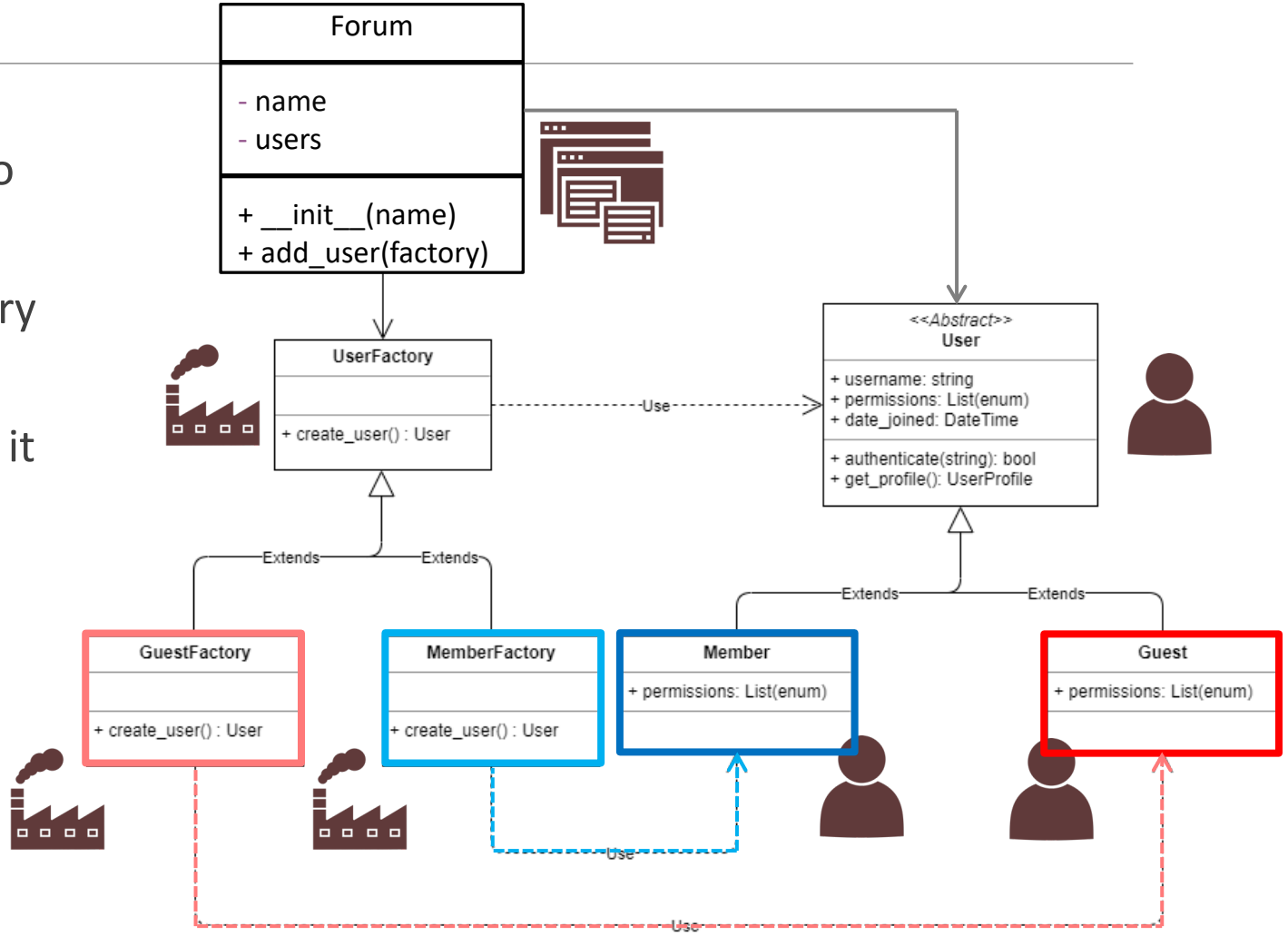
Step 4:
Integrate it into our Forum Website

We can have a ForumController that prompts the user for the type of user to create

And that's it! We have a working Factory Pattern.

It's pretty straight forward if you break it down right?

forum_user_factory.py



Factory Pattern:
Why and When do we use it

- Use this pattern when you want to **separate and encapsulate the creation process**.
- When we want to provide **a separate interface for object creation** where each factory **subclass can alter the type of object created.**
- Adheres to the Single Responsibility, Open Closed, Liskov Substitution and Dependency Inversion Principle.
- Use the Factory Pattern when the **exact type and dependencies of the objects you need are unknown or susceptible to change**. If we want to add a new product, all we need to do is create a new Factory and override the creation method in it.

Factory Pattern – Disadvantages

- There are a **lot of classes** in play. This can make the code complex and hard to debug.
- Sometimes the classes can get artificial. You may decide to **create a whole new subclass for a very minor change** in the object creation process.

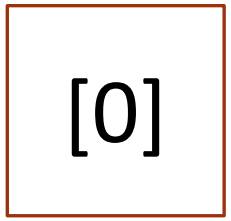
Function parameters

PASS BY VALUE? PASS BY REFERENCE? OR SOMETHING ELSE???

Pass by value

Check out this code, and let's assume it's **pass by value**

```
def func(in_list):
    in_list.append(1)
    in_list = [2, 3]

out_list = [0]
func(out_list)
print(out_list)
```



out_list

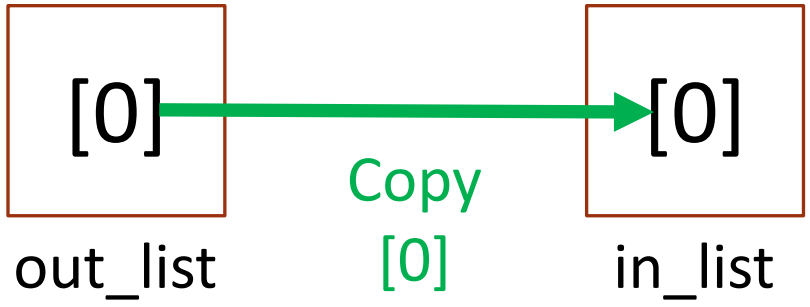
A new object is created in memory, and the variable out_list is assigned to it

http://stupidpythonideas.blogspot.com/2013/11/does-python-pass-by-value-or-by.html

Pass by value

Check out this code, and let's assume it's pass by value

```
def func(in_list):
    in_list.append(1)
    in_list = [2, 3]

out_list = [0]
func(out_list)
print(out_list)
```



in_list is a new parameter. A new piece of memory is created and a copy of [0] is placed into the memory of in_list

Pass by value

Check out this code, and let's assume it's pass by value

```
def func(in_list):
    in_list.append(1)
    in_list = [2, 3]

out_list = [0]
func(out_list)
print(out_list)
```

in_list has the value 1 appended to it. This does not change the original out_list

Pass by value

Check out this code, and let's assume it's pass by value

```
def func(in_list):
    in_list.append(1)
    in_list = [2, 3]

out_list = [0]
func(out_list)
print(out_list)
```

in_list has now changed its value to [2,3]

Pass by value

Check out this code, and let's assume it's pass by value

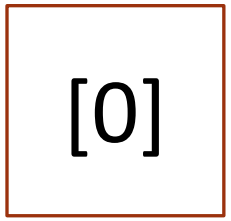
```
def func(in_list):
    in_list.append(1)
    in_list = [2, 3]

out_list = [0]
func(out_list)
print(out_list)
```

When we print out_list, its value still remains as **[0]**

Pass by reference

Check out this code, and let's assume it's **pass by reference**

```
def func(in_list):
    in_list.append(1)
    in_list = [2, 3]

out_list = [0]
func(out_list)
print(out_list)
```



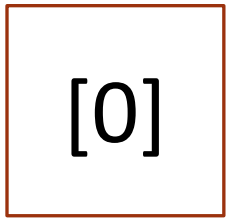
out_list

A new object is created in memory, and the variable out_list is assigned to it

Pass by reference

Check out this code, and let's assume it's pass by reference

```
def func(in_list):
    in_list.append(1)
    in_list = [2, 3]

out_list = [0]
func(out_list)
print(out_list)
```



out_list
in_list

in_list is a reference to out_list. in_list and out_list are both referencing the same object in memory. There is NO COPYING

Pass by reference

Check out this code, and let's assume it's pass by reference

```
def func(in_list):
    in_list.append(1)
    in_list = [2, 3]

out_list = [0]
func(out_list)
print(out_list)
```

out_list
in_list

in_list has the value 1 appended to it. This changes the original value out_list is assigned to

Pass by reference

Check out this code, and let's assume it's pass by reference

```
def func(in_list):
    in_list.append(1)
    in_list = [2, 3]

out_list = [0]
func(out_list)
print(out_list)
```

out_list
in_list

in_list has now changed its value to [2,3]. Again this changes the original value out_list is assigned to

Pass by reference

Check out this code, and let's assume it's pass by reference

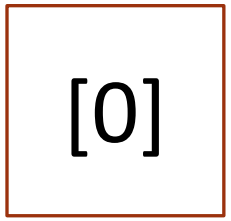
```
def func(in_list):
    in_list.append(1)
    in_list = [2, 3]

out_list = [0]
func(out_list)
print(out_list)
```

out_list
in_list

When we print out_list, its value was changed within the function to **[2,3]**. These changes are reflected even outside the function

Python - Pass by ???

```
def func(in_list):
    in_list.append(1)
    in_list = [2, 3]

out_list = [0]
func(out_list)
print(out_list)
```

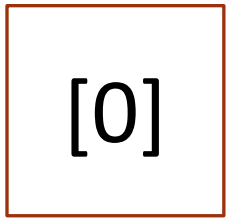


out_list

A new object is created in memory, and the variable out_list is assigned to it

Python - Pass by ???

```
def func(in_list):
    in_list.append(1)
    in_list = [2, 3]

out_list = [0]
func(out_list)
print(out_list)
```



out_list
in_list

in_list is a new parameter that points to the same value out_list is pointing at. in_list and out_list are both pointing at the same object in memory. There is NO COPYING

Python - Pass by ???

```
def func(in_list):
    in_list.append(1)
    in_list = [2, 3]

out_list = [0]
func(out_list)
print(out_list)
```

out_list
in_list

in_list has the value 1 appended to it. This changes the original value out_list is assigned to

Python - Pass by ???

```
def func(in_list):
    in_list.append(1)
    in_list = [2, 3]

out_list = [0]
func(out_list)
print(out_list)
```

The object [2,3] is created in memory and in_list has now changed to point to it

Python - Pass by ???

```
def func(in_list):
    in_list.append(1)
    in_list = [2, 3]

out_list = [0]
func(out_list)
print(out_list)
```

When we print out_list, its value is **[0,1]**

Python - Pass by ???

Soooo, what is it called?

You might see some of the following ways to call it:

Call-by-Object
Call by Object Reference
Call by Sharing
Pass-by-object-reference
Pass by assignment
Pass by name

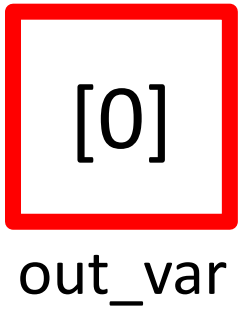
Some people don't bother giving it a name because it's an entirely different concept from pass by value or pass by reference

Mutable vs Immutable Function parameters

MORE PYTHON QUIRKS

Mutable function parameter

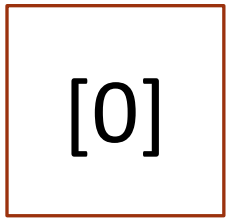
```
def my_func(in_var):
    in_var.append(1)
    in_var = [2, 3]

out_var = [0]
my_func(out_var)
print(out_var)
```



A **new list object** is created in memory, and the variable out_var is assigned to it

Mutable function parameter

```
def my_func(in_var):
    in_var.append(1)
    in_var = [2, 3]

out_var = [0]
my_func(out_var)
print(out_var)
```



out_var
in_var

in_var is a new variable that points to the same value out_var is pointing at. in_var and out_var are both pointing at the same object in memory. There is NO COPYING

Mutable function parameter

```
def my_func(in_var):
    in_var.append(1)
    in_var = [2, 3]

out_var = [0]
my_func(out_var)
print(out_var)
```

out_var
in_var

in_var has the value 1 appended to it. This changes the original value out_var is assigned to

Mutable function parameter

```
def my_func(in_var):
    in_var.append(1)
    in_var = [2, 3]

out_var = [0]
my_func(out_var)
print(out_var)
```

A **new list object** [2,3] is created in memory and in_var has now changed to point to it

Mutable function parameter

```
def my_func(in_var):
    in_var.append(1)
    in_var = [2, 3]

out_var = [0]
my_func(out_var)
print(out_var)
```

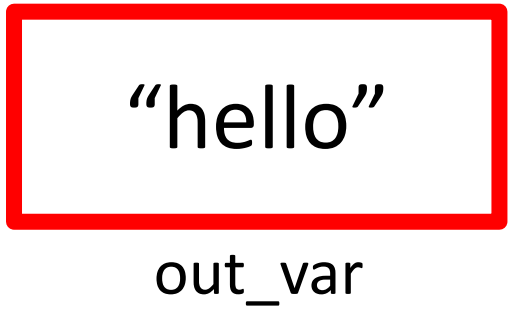
When we print out_var, its value is [0,1]

Mutable parameters appear to treat function parameters *somewhat like* pass by reference. Changes made internally to the mutable parameter change the original object

However assignment (=) causes the parameter to point to different objects

Immutable function parameter

```
def my_func(in_var):
    in_var += “world”
    in_var = “bye”

out_var = “hello”
my_func(out_var)
print(out_var)
```



A **new immutable string object** is created in memory, and the variable out_var is assigned to it

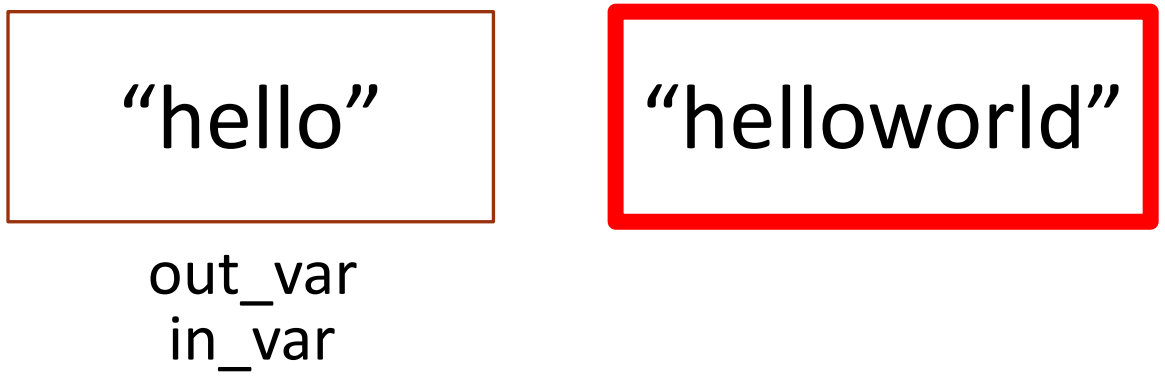
Immutable function parameter

```
def my_func(in_var):
    in_var += “world”
    in_var = “bye”

out_var = “hello”
my_func(out_var)
print(out_var)
```

“hello” is passed as a parameter to in_var. Before executing the first line, in_var and out_var are both pointing at the same “hello” object

Immutable function parameter

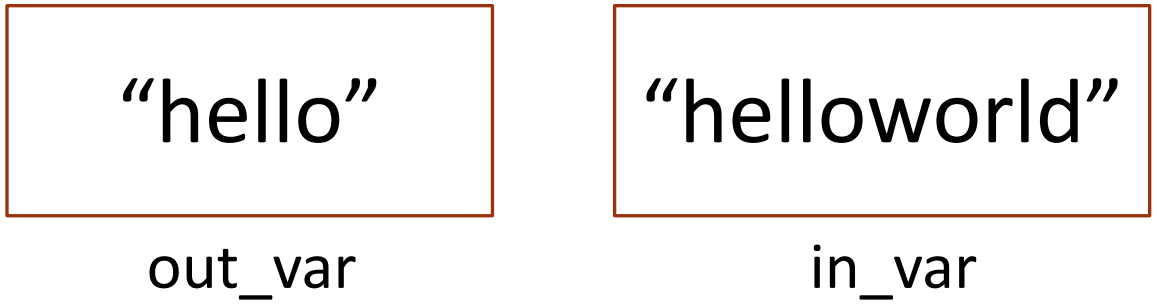
```
def my_func(in_var):
    in_var += “world”
    in_var = “bye”

out_var = “hello”
my_func(out_var)
print(out_var)
```



Strings are immutable. Meaning that we can’t change a string in memory that already exists
Appending “world” to “hello” requires creating an entirely **new string object**

Immutable function parameter

```
def my_func(in_var):
    in_var += “world”
    in_var = “bye”

out_var = “hello”
my_func(out_var)
print(out_var)
```



Assigning “helloworld” to in_var, makes in_var point to the new “helloworld” object

Immutable function parameter

```
def my_func(in_var):
    in_var += “world”
    in_var = “bye”

out_var = “hello”
my_func(out_var)
print(out_var)
```

“bye” is an entirely **new string object**.
Memory is allocated for “bye” and in_var is pointing at it

Immutable function parameter

```
def my_func(in_var):
    in_var += “world”
    in_var = “bye”

out_var = “hello”
my_func(out_var)
print(out_var)
```

out_var is still pointing at the string “hello” so it prints out “hello”

Immutable parameters make the function appear to treat parameters as pass by value

Mutable vs immutable parameters summary

Be aware of what types of parameters are passed into a function

Mutable parameters

- Can make functions appear ***somewhat like*** **pass by reference**
- Can change the internal attributes
- Changes will affect the original object
- Assignment causes parameter to point to a different object. Causes function to appear to be pass by value

Immutable parameters

- Can make functions appear to be **pass by value**
- Can NOT change the internal attributes. This is consistent with the concept of immutable types
- Any changes that appear to occur are actually new immutable objects that are created